7. Prototípus koncepciója

48 – GonoszOnosz

Konzulens:

Marcsingó Ádám

Csapattagok

Kálmán Patrik	XG5YQ1	kalmanpatrik1214@gmail.com
Papp Bálint	HND2AH	xypapbal@gmail.com
dr. Grób László Mihály	EAS7U0	laszlo.m.grob@gmail.com
Héja Gergő Zoltán	FEFSRB	hejagergo@gmail.com
Dobronyi Benedek Márk	IMQ1AR	benedek.dobronyi@gmail.com

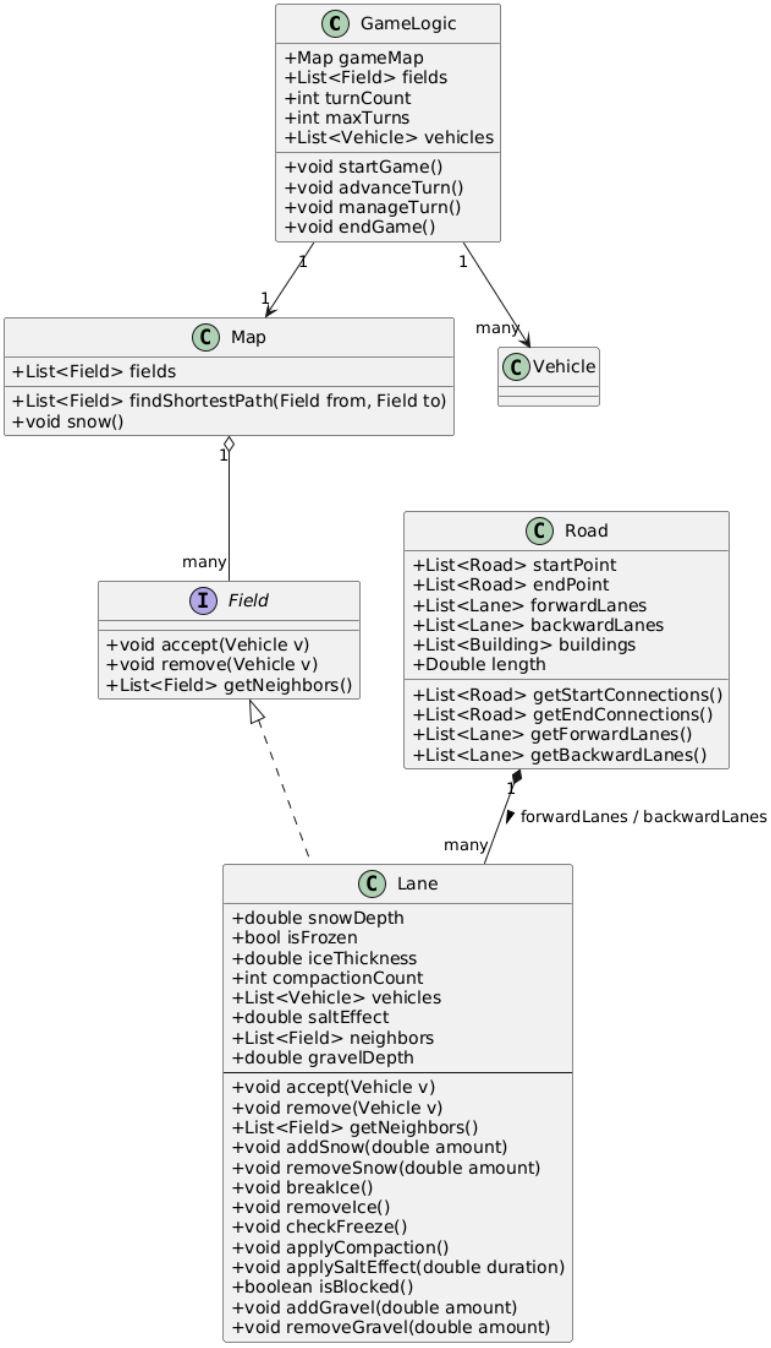
2026.04.10

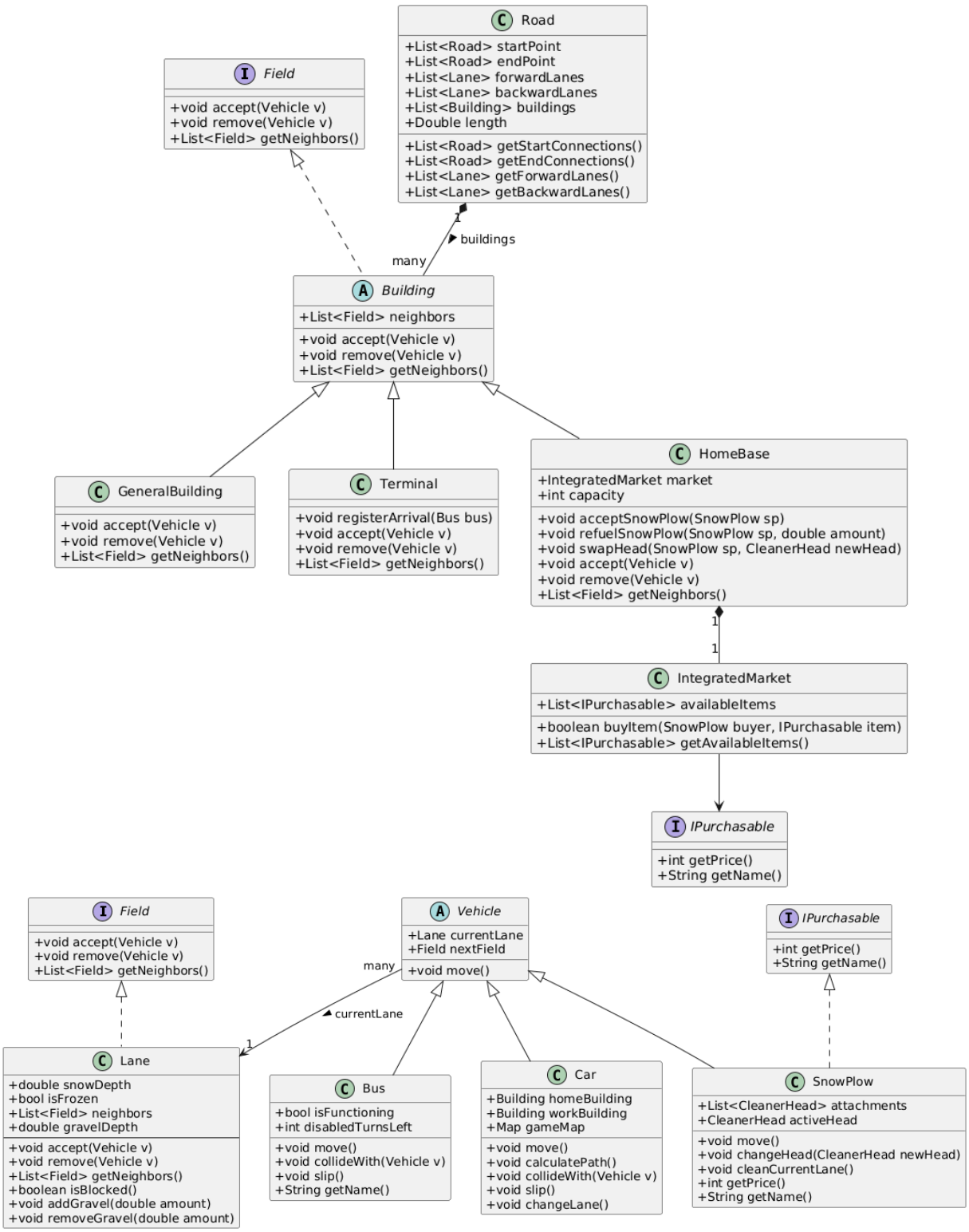
7. Prototípus koncepciója

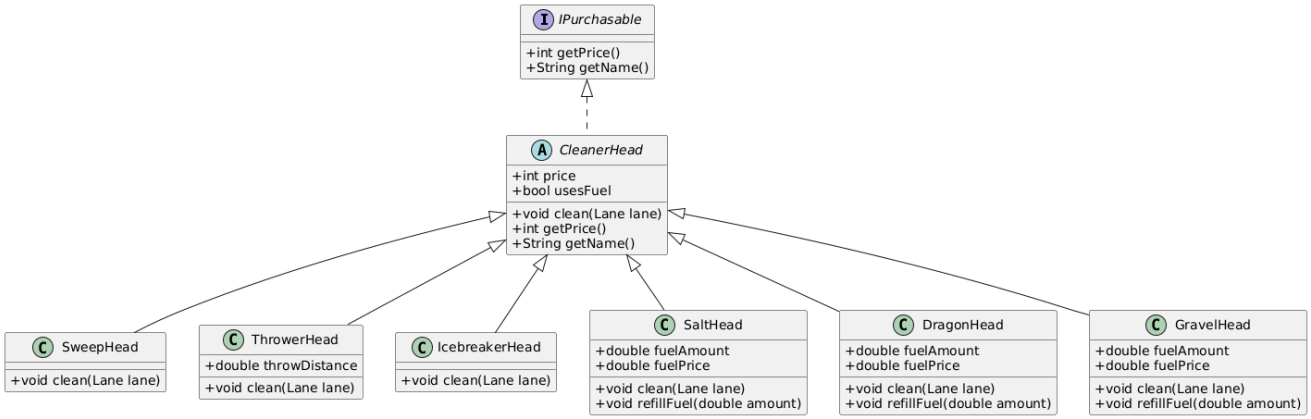
7.0 Változás hatása a modellre

A változás szövege: *A hókotrókra zúzott követ (zuzalékot) szóró fejet is lehet szerelni. A zúzott kő a sóhoz hasonlóan szerezhető be, egy hókotrón csak véges mennyiség fér el egyidőben. A kiszórt zuzalék a jég csúszósságát megszünteti, de a söprő és hányó fej a zuzalékot ugyanúgy eltakarítja, mint a havat. A lángszóró, a jégtörő és a só a zuzalékra nem hat. A zuzalék nem tömörödik. Ha hó esik rá, akkor a hó egy idő után befedi, a hó a szokott módon letaposható és így lefagyott jéggé válik.*

7.0.1 Módosult osztálydiagram




Road
+List<Road> startPoint
+List<Road> endPoint
+List<Lane> forwardLanes
+List<Lane> backwardLanes
+List<Building> buildings
+Double length
+List<Road> getStartConnections()
+List<Road> getEndConnections()
+List<Lane> getForwardLanes()
+List<Lane> getBackwardLanes()
Field
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
1
buildings
many
Building
+List<Field> neighbors
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
GeneralBuilding
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
Terminal
+void registerArrival(Bus bus)
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
HomeBase
+IntegratedMarket market
+int capacity
+void acceptSnowPlow(SnowPlow sp)
+void refuelSnowPlow(SnowPlow sp, double amount)
+void swapHead(SnowPlow sp, CleanerHead newHead)
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
1
1
IntegratedMarket
+List<IPurchasable> availableItems
+boolean buyItem(SnowPlow buyer, IPurchasable item)
+List<IPurchasable> getAvailableItems()
IPurchasable
+int getPrice()
+String getName()
Field
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
Vehicle
+Lane currentLane
+Field nextField
+void move()
IPurchasable
+int getPrice()
+String getName()
many
currentLane
1
Lane
+double snowDepth
+bool isFrozen
+List<Field> neighbors
+double gravelDepth
+void accept(Vehicle v)
+void remove(Vehicle v)
+List<Field> getNeighbors()
+boolean isBlocked()
+void addGravel(double amount)
+void removeGravel(double amount)
Bus
+bool isFunctioning
+int disabledTurnsLeft
+void move()
+void collideWith(Vehicle v)
+void slip()
+String getName()
Car
+Building homeBuilding
+Building workBuilding
+Map gameMap
+void move()
+void calculatePath()
+void collideWith(Vehicle v)
+void slip()
+void changeLane()
SnowPlow
+List<CleanerHead> attachments
+CleanerHead activeHead
+void move()
+void changeHead(CleanerHead newHead)
+void cleanCurrentLane()
+int getPrice()
+String getName()



7.0.2 Új vagy megváltozó metódusok

Pirossal jelölve az új, vagy megváltozott részeket

Lane (Sáv)

Felelősség

Az út egyetlen forgalmi sávja és egyben Field a gráfban. Felelőssége nyilvántartani a hóréteg vastagságát, a jégpáncél állapotát és a rajta tartózkodó járműveket. Kezeli a letaposás miatti jégképződési mechanizmust.

Ősosztályok

Nincs

Interfészek

Field

Attribútumok

- double snowDepth: a sávon lévő hóréteg vastagsága
- bool isFrozen: igaz, ha a sávon jégpáncél van
- double iceThickness: a jégpáncél vastagsága
- int compactionCount: hányszor hajtottak át járművek – küszöb felett jégpáncél képződik
- List<Vehicle> vehicles: a sávon tartózkodó járművek listája
- double saltEffect: a sóhatás fennmaradásának ideje
- List<Field> neighbors: a szomszédos mezők (Field interfészből)
- double gravelDepth: a sávon lévő zúzalékréteg vastagsága

Metódusok

- void accept(Vehicle v): fogadja az érkező járművet (Field-ből felüldefiniálva)
- void remove(Vehicle v): eltávolítja a távozó járművet (Field-ből felüldefiniálva)
- List<Field> getNeighbors(): visszaadja a szomszédos mezőket (Field-ből)
- void addSnow(double amount): növeli a hóréteg vastagságát
- void removeSnow(double amount): csökkenti a hóréteg vastagságát
- void breakIce(): a jégpáncélt feltöri és havává alakítja
- void removeIce(): eltávolítja a jégpáncélt
- void checkFreeze(): ellenőrzi, hogy kell-e jégpáncél képződjön. Bár a jég kialakulhat, a csúszósságot a gravelDepth felülírhatja
- void applyCompaction(): növeli a compactionCount-ot járműáthaladáskor
- void applySaltEffect(double duration): beállítja a sóhatást a sávon
- boolean isBlocked(): igaz, ha a sáv járhatatlan

- void addGravel(double amount): növeli a sávon lévő zúzalék mennyiségét
- void removeGravel(double amount): csökkenti a sávon lévő zúzalék mennyiségét

GravelHead (Zúzalékszóró fej)

Felelősség

Zúzalékot (zúzott követ) szóró hókotró fej. A kiszórt zúzalék megszünteti a jég csúszósságát, de a hó befedheti. Működéséhez zúzalék (mint üzemanyag) szükséges; ha a készlet kiürül, a clean() nem hat.

Ősosztályok

CleanerHead

Interfészek

Nincs

Attribútumok

- double fuelAmount: a jelenleg tárolt zúzalék mennyisége
- double fuelPrice: a tárolt üzemanyag (zúzalék) ára

Metódusok

- void clean(Lane lane): zúzalékot juttat a sávra (addGravel), megakadályozva a csúszásveszélyt, és felhasználva a zúzalékból a szükséges mennyiséget (CleanerHead-ből felüldefiniálva)
- void refillFuel(double amount): zúzalékot tölt a tartályba a telephelyen

SweepHead (Söprő fej)

Felelősség

A hókotróra szerelhető söprő típusú fej. A havat a szomszédos sávba vagy az út mellé tolja. Jeget nem tud eltávolítani. Üzemanyagot nem igényel.

Ősosztályok

CleanerHead

Interfészek

Nincs

Attribútumok

–

Metódusok

- void clean(Lane lane): a sáv havát és zúzalékát áttol a szomszédos sávba vagy az út szélére, jégre nincs hatása

ThrowerHead (Hányó fej)

Felelősség

A söprőhöz hasonló hókotró fej, de a havat jóval messzebbre (pl. a járdára) szórja, nem csupán a szomszédos sávba. Jeget nem tud eltávolítani. Üzemanyagot nem igényel.

Ősosztályok

CleanerHead

Interfészek

Nincs

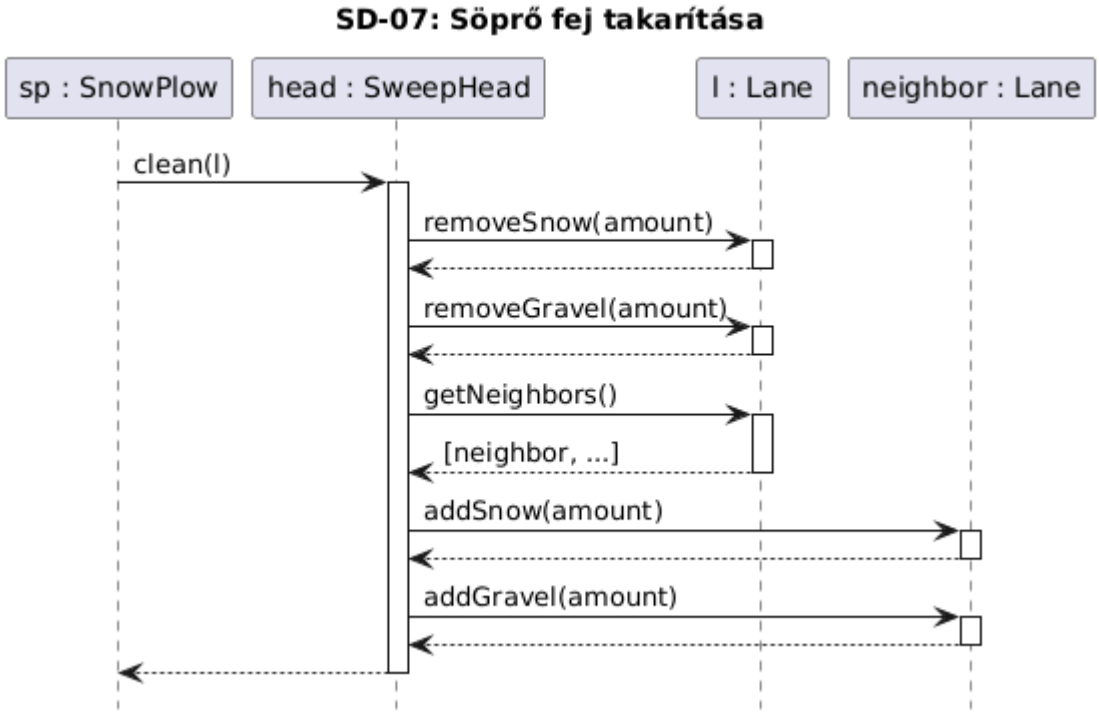
Attribútumok

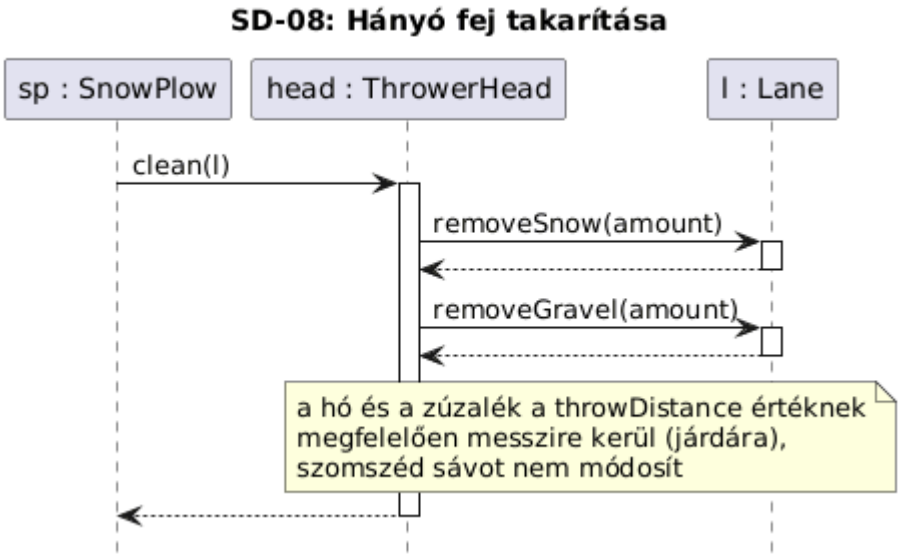
- double throwDistance: az a távolság, ameddig a hányó fej a havat elhajítja

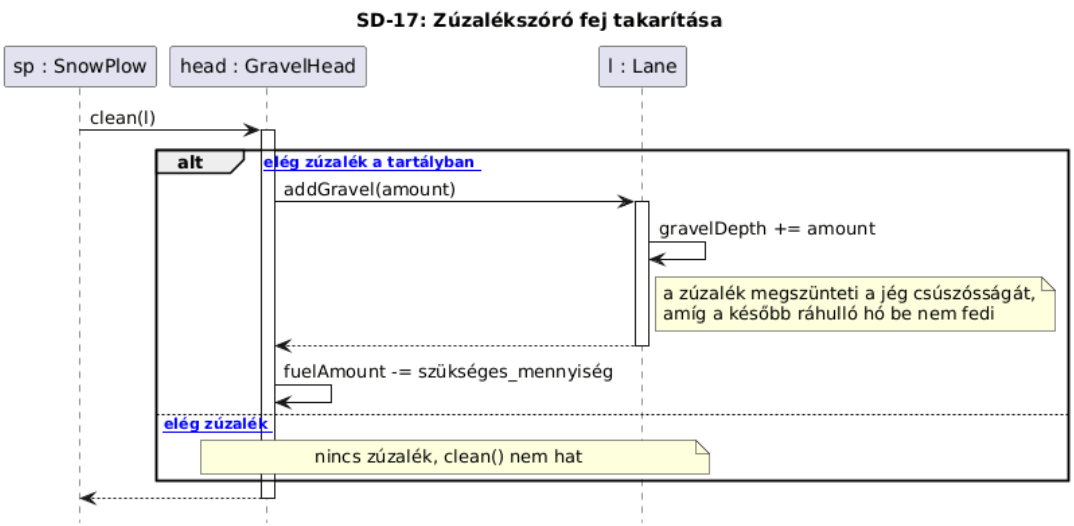
Metódusok

- void void clean(Lane lane): a sáv havát és zúzalékát messzire szórja el az útról, a szomszédos sávokat nem módosítja; jégre nincs hatása

7.0.3 Szekvencia-diagramok







7.1 Prototípus interface-definíciója

7.1.1 Az interfész általános leírása

A prototípus fázisban a szoftver egy tisztán parancssoros (CLI – Command Line Interface) alkalmazásként működik, grafikus felhasználói felület (GUI) nélkül. Az interfész kialakításának elsődleges célja nem a felhasználói élmény maximalizálása, hanem az analízis modellben definiált játéklogika, a belső állapotgépek és az objektumok közötti interakciók robusztus, automatizálható tesztelhetőségének biztosítása. Ennek érdekében az interfész az alábbi alapelvekre épül:

- **Szabványos I/O csatornák használata:** A felhasználóval, illetve a tesztelő környezettel való kommunikáció szigorúan és kizárólag a szabványos bemeneten (Standard Input) és a szabványos kimeneten (Standard Output) keresztül történik. A program nem használ platformfüggő konzolos megszakításokat, interaktív menüket vagy speciális képernyőtörlő/pozicionáló parancsokat, így a bemenet egy az egyben fájlból is átirányítható.
- **Szekvenciális parancsfeldolgozás:** A program interaktív parancsértelmezőként (shell) működik. A bemeneten soronként várja az utasításokat, azokat szintaktikailag és szemantikailag elemzi, végrehajtja a belső modellben, majd az eredményt (vagy a hibaüzenetet) a szabványos kimenetre írja, egészen a futást megszakító parancs beérkezéséig, vagy a bemeneti fájl végéig (EOF).
- **Determinisztikus működés és tesztelhetőség:** Mivel az üzleti logika tartalmazhat véletlenszerű elemeket (pl. a sávok jéggé fagyásának pontos pillanata, vagy a megcsúszás bekövetkezése), egy fix *seed* megadására. Ez garantálja, hogy egy adott bemeneti fájl minden egyes futtatáskor bitre pontosan ugyanazt a kimenetet eredményezi, ami az automatizált tesztelés alapfeltétele.
- **Állapotvezérelt indítás és lekérdezés:** Az inicializálási parancsok segítségével a rendszer bármilyen komplex, előre beállított szituációból (konkrét pályakép, specifikus hóvastagság, elakadt járművek) elindítható anélkül, hogy a játékot az elejétől végig kellene játszani. A végrehajtott műveletek hatásának ellenőrzésére pedig olyan strukturált, könnyen olvasható kimenetet generáló szolgálnak, amelyekkel a belső objektumok rejtett állapotai is transzparensen a konzolra logolhatók. A kimenet formátuma prediktabilis, így automatikus szöveg-összehasonlító eszközökkel könnyen validálható.

7.1.2 Bemeneti nyelv

Általános szabályok (Szintaxis és Szemantika)
Minden parancs egy parancsszóval kezdődik, melyet a paraméterek követnek, szóközzel elválasztva.
A rendszer minden entitása (sáv, épület, jármű, fej) egyedi string azonosítóval (ID) rendelkezik (pl. lane_1).
A megjegyzések (kommentek) a sor elején elhelyezett # karakterrel kezdődnek. A parancsértelmező ezeket a sorokat figyelmen kívül hagyja.
A parancsok végrehajtása azonnal megtörténik a belső modellben.
Parancsok listája

- **random**
  - **Leírás:** Beállítja a játék determinisztikusságát. Kikapcsolt állapotban az esélyalapú események (pl. csúszás a jégen) garantáltan bekövetkeznek a tesztesetek bejárhatósága érdekében.
  - **Opciók:** [on|off] (pl. random off)
- **force_slip**
  - **Leírás:** Determinisztikus módban (random off) használható. Ha egy jármű jégre fut, alapesetben mindig megcsúszna determinisztikus módban. Ezzel a paranccsal explicitly kikényszeríthető (vagy megelőzhető), hogy egy specifikus jármű megcsússzon, ezzel tesztelve az ütközés vagy az elkerülés ágait.
  - **Opciók:** <vehicle_id> [true|false] (pl. force_slip car_1 true)
- **next_turn**
  - **Leírás:** Szimulálja az idő múlását. Meghívja a GameLogic.advanceTurn() metódust. Ekkor hajtódnak végre a rendszer-események: havazik, az NPC autók lépnek a saját útvonalkeresőjük alapján, és a sóhatás csökken.
  - **Opciók:** [körök száma] – Opcionális paraméter. Ha meg van adva, a rendszer ennyi kört szimulál le egyszerre (alapértelmezetten 1).
- **load**
  - **Leírás:** Inicializálja a játékteret egy megadott konfigurációs fájl alapján. Törli a jelenlegi állapotot, és betölti az új térképet, járműveket és statisztikákat.
  - **Opciók:** <fájlnév> (pl. load test_map_01.txt)
- **move_bus**
  - **Leírás:** A buszvezető játékos akciója. Utasítja a megadott azonosítójú buszt, hogy a szomszédos célmezőre mozogjon a körében.
  - **Opciók:** <bus_id> <target_field_id> (pl. move_bus bus_1 lane_2)
- **move_plow**
  - **Leírás:** A takarító játékos akciója. Utasítja a megadott hókotrót a következő mezőre lépésre. A lépés után automatikusan meghívódik az aktív tisztítófej clean() metódusa az új sávon.
  - **Opciók:** <plow_id> <target_field_id>
- **buy**
  - **Leírás:** A takarító játékos akciója a telephelyen (HomeBase). A megadott hókotró számára vásárol egy új tisztítófejet vagy üzemanyagot (sót, biokerozint, zúzalékot) a beépített áruházból.
  - **Opciók:** <player_id> <plow_id> <item_type> [amount] (pl. buy cl1 sp_1 gravelhead vagy buy sp_1 gravel 50)
- **equip**
  - **Leírás:** A takarító játékos akciója a telephelyen. Lecseréli a hókotró aktív tisztítófejét egy már megvásárolt (az attachments listában szereplő) fejre.
  - **Opciók:** <plow_id> <head_id>

- **stat**
  - **Leírás:** A tesztelés kimenet-generáló parancsa. Lekérdezi és a kimenetre írja egy adott objektum belső állapotát (változóit). A válasz formátumát a Kimeneti nyelv definiálja.
  - **Opciók:** <objektum_id> (pl. stat lane_1)
- **list**
  - **Leírás:** Kiírja az összes adott típusú objektum ID-ját
  - **Opciók:** <type>
- **stat**
  - **Leírás:** Kiírja a lekérdezett objektum összes releváns attribútumát.
  - **Opciók:** <id>
- **save**
  - **Leírás:** A teljes jelenlegi memóriaképet kimenti a szabványos kimenetre (vagy egy fájlba) olyan formátumban, ami később a load paranccsal visszatölthető.
  - **Opciók:** [fájlnév] (opcionális, ha nincs, a standard outputra ír).
- **exit**
  - **Leírás:** Leállítja a prototípus futását és bezárja az I/O csatornákat.
  - **Opciók:** Nincs.

Térképépítő és Konfigurációs parancsok (A load által is használt szintaxis)

Ezek a parancsok felelnek a játék kezdeti állapotának felépítéséért és a determinisztikus szituációk előállításáért.

- **create**
  - **Leírás:** Létrehoz egy új, paraméterezetlen objektumot a memóriában a megadott azonosítóval.
  - **Opciók:** <típus> <id>
  - **Példák:** * create road r1
    - create lane l1
    - create homebase hb1
    - create building b1
    - create terminal t1
    - create cleaner c_1
    - create busdriver bd_1

- **add_to_road**
  - **Leírás:** Kialakítja a tartalmazási (kompozíciós) hierarchiát a modellen. Hozzárendel egy már létrehozott sávot vagy épületet egy meglévő úthoz (Road). Sáv hozzáadása esetén kötelező megadni az irányt, hogy a Road a forwardLanes vagy backwardLanes listájába tegye . Épület esetén az irány paraméter elhagyható.
  - **Opciók:** <road_id> <item_type> <item_id> [direction]
  - **Példák:**
    - add_to_road r1 lane l1 forward
    - add_to_road r1 lane l2 backward
    - add_to_road r1 building b1
- **connect_roads**
  - **Leírás:** A magas szintű logikai úthálózatot köti össze. Beállítja, hogy az egyik út melyik vége a másik út melyik végéhez csatlakozik (a Road startPoint és endPoint listáit tölti fel) .
  - **Opciók:** <road1_id> <pos1> <road2_id> <pos2> (ahol a pos lehet start vagy end).

    - **Példa:** connect_roads r1 end r2 start (Az r1 út vége csatlakozik az r2 út elejéhez).
  - **connect_fields**
    - **Leírás:** A navigációs gráf éleit hozza létre. Két Field interfészt megvalósító objektumot köt össze (pl. Lane -> Lane, vagy Building -> Lane), hozzáadva őket a neighbors listához .
    - **Opciók:** <field1_id> <field2_id>
    - **Példák:**
      - connect_fields l1 l2.

set_road_length
- **Leírás:** Beállítja egy út fizikai/matematikai hosszát, amely az útvonalkeresés (súlyozás) és a sáv-kapacitások alapja.
- **Opciók:** <road_id> <length>
- **Példa:** set_road_length r1 150.0

set_lane_state
- **Leírás:** Fizikai értékeket (hóvastagság, jég, zúzalék) állít be egy sávon, hogy specifikus teszthelyzeteket lehessen szimulálni.
- **Opciók:** <lane_id> <snowDepth> <isFrozen> <iceThickness> <gravelDepth>
- **Példa:** set_lane_state l1 15.0 false 0.0 5.0

spawn
- **Leírás:** Járművet helyez el a pályán egy adott csomópontban (Field), valamint beállítja a paramétereit (pl. egy autó otthonát és célját).
- **Opciók:**
  - Hókotró: spawn snowplow <id> <field_id>
  - Autó: spawn car <id> <field_id> <home_building_id> <work_building_id>
  - Busz: spawn bus <id> <field_id>

set_money
- **Leírás:** Beállítja a játékosok kezdőtőkéjét.
- **Opciók:** <player_id> <amount> (ahol player_type: cleaner vagy busdriver).
- **Példa:** set_money cleaner 5000

set_score
- **Leírás:** Beállítja a játékosok aktuális pontszámát.
- **Opciók:** <player_id> <amount> (ahol player_type: cleaner vagy busdriver).
- **Példa:** set_money cleaner 5000

7.1.3 Kimeneti nyelv

1. Általános kimeneti elemek és szintaxis

[SUCCESS] – Sikeres akciók nyugtázása Minden érvényes, állapotváltozást okozó direkt parancs (pl. move_bus, buy, create, spawn) egy nyugtázó üzenetet generál.
- *Szintaxis:* [SUCCESS] <ObjektumID> <Akció> <Eredmény>
- *Példák:*
  - [SUCCESS] sp_1 moved to l_2
  - [SUCCESS] c_1 bought gravelhead for sp_1
  - [SUCCESS] lane l_1 created

[ERROR] – Hibakezelés Érvénytelen szintaxis, nem létező ID, vagy a játékszabályok megsértése esetén (pl. nincs elég pénz, járhatatlan sáv).
- *Szintaxis:* [ERROR] <Hiba_oka>
- *Példák:*
  - [ERROR] Invalid field ID: l_99
  - [ERROR] Not enough money for player c_1

    - [ERROR] Vehicle car_1 cannot move, lane l_2 is blocked

[EVENT] – Rendszeresemények (A next_turn hatására) A körök léptetésekor bekövetkező aszinkron és logikai események (NPC-k mozgása, havazás, csúszás) logolása.

- *Példák:*
    - [EVENT] Map snowed. Global depth increased.
    - [EVENT] car_1 moved to l_3
    - [EVENT] car_2 slipped on l_4. Field is now blocked.
    - [EVENT] bus_1 completed a round at t_1.

2. Állapotlekérdező kimenetek (stat és list)

A tesztelők a stat és list parancsokkal kérhetik le a belső memóriaképet. Ennek kimenete soronként strukturált, kulcs-érték párokat tartalmazó blokk.

[LIST] – Objektumok felsorolása A list <type> parancs kimenete.

- *Szintaxis:* [LIST] <Típus>: <ID_1>, <ID_2>, ...
- *Példa:* [LIST] vehicles: car_1, bus_1, sp_1

[STATE] – Belső állapot dump A stat <ID> parancs kimenete. Kiírja a lekérdezett objektum összes releváns attribútumát.

- *Szintaxis:* text [STATE] <ID> (<Osztály>) <attribútum1>:<érték><attribútum2>:<érték>

3. Teljes állapotmentés kimenete (save)

- A program fel van készítve a teljes aktuális modellállapot szöveges mentésére (pl. a save parancs kiadásakor). A mentés szintaxisa egyszerű: **a kimenet 100%-ban megegyezik a Térképépítő és Konfigurációs bemeneti nyelvvel** (lásd 7.1.2).
- A kimenet minden sora egy érvényes create, connect_fields, set_lane_state, spawn, vagy set_money parancs, amely sorra veszi a memóriában lévő objektumokat.
- Az így generált kimeneti fájl (*.txt) közvetlenül felhasználható egy új load parancs bemeneteként, így a tesztek bármikor megszakíthatók és később onnan folytathatók, valamint az elvárt állapot (expected output) könnyen megírható a teszteléshez.

7.2 Összes részletes use-case

Use-case neve	1. Játéktér betöltése
Rövid leírás	A tesztelő betölt egy előre megírt konfigurációs fájlt.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő beírja a load <fájlnév> parancsot. A rendszer betölti a pályát és [SUCCESS] üzeneteket ír ki.

Use-case neve	2. Pálya mentése
Rövid leírás	A jelenlegi állapot kimentése fájlba.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő kiadja a save parancsot. A rendszer inicializáló parancsok formájában kilogolja a pálya állapotát.

Use-case neve	3. Determinisztikus mód bekapcsolása
Rövid leírás	A véletlenszerűség kikapcsolása a kiszámítható tesztekhez.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő kiadja a random off parancsot. A rendszer [SUCCESS] üzenettel nyugtázza a beállítást.

Use-case neve	4. Megcsúszás kikényszerítése
Rövid leírás	Determinisztikus módban egy jármű garantáltan megcsúszik a jégen.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő beírja a force_slip <id> true parancsot. A rendszer nyugtázza a kérést.

Use-case neve	5. Entitások listázása
Rövid leírás	A memóriában lévő objektumok lekérdezése típus szerint.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő beírja a list vehicles parancsot. A rendszer kilistázza az összes jármű azonosítóját ([LIST]).

Use-case neve	6. Állapot lekérdezése
Rövid leírás	Egy konkrét objektum belső paramétereinek kiíratása.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő beírja a stat <id> parancsot. A rendszer kiírja az objektum összes adatát egy [STATE] blokkban.

Use-case neve	7. Kör léptetése (1x)
Rövid leírás	Az idő múlásának és az NPC-k lépésének szimulálása.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő kiadja a next_turn parancsot. A kimeneten megjelennek a bekövetkezett események ([EVENT]).

Use-case neve	8. Körök léptetése (Nx)
Rövid leírás	Több kör eseményeinek egyidejű leszimulálása.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő beírja a next_turn n parancsot. A rendszer egymás után n kör eseményeit listázza ki.

Use-case neve	9. Játékos pénzének módosítása
Rövid leírás	Egyedi kezdőösszeg.
Aktorok	Tesztelő
Forgatókönyv	A tesztelő beírja a set_money cleaner 10000 parancsot. A rendszer [SUCCESS] üzenettel nyugtázza az átírást.

Use-case neve	10. Busz mozgatása szabad sávra
Rövid leírás	A buszvezető egy szomszédos, szabad sávra lép.
Aktorok	Buszvezető
Forgatókönyv	A buszvezető beírja a move_bus <bus_id> <lane_id> parancsot. A rendszer [SUCCESS] üzenetet ad.

Use-case neve	11. Busz mozgatása végállomásra
Rövid leírás	A busz eléri a célját, teljesítve egy fordulót.
Aktorok	Buszvezető
Forgatókönyv	A buszvezető a buszt egy Terminal mezőre lépteti. A rendszer [EVENT] üzenettel jelzi a forduló teljesítését.

Use-case neve	12. Hókotró mozgatása
Rövid leírás	A takarító egy szomszédos sávra lép a munkagéppel.
Aktorok	Takarító
Forgatókönyv	A takarító beírja a move_plow <plow_id> <lane_id> parancsot. A rendszer [SUCCESS] üzenetet ad, a sáv paraméterei frissülnek.

Use-case neve	13. Hókotró fej vásárlása
Rövid leírás	Új tisztítófej vétele a telephelyen.
Aktorok	Takarító
Forgatókönyv	A takarító beírja a buy <player_id> <plow_id> <head_type> parancsot. A rendszer levonja a pénzt és nyugtázza a tranzakciót.

Use-case neve	14. Üzemanyag vásárlása
Rövid leírás	Zúzalék, só vagy biokerozin vétele a gépbe.
Aktorok	Takarító
Forgatókönyv	A takarító beírja a buy <player_id> <plow_id> gravel 50 parancsot. A rendszer növeli a készletet és [SUCCESS]-t jelez.

Use-case neve	15. Tisztítófej cseréje
Rövid leírás	Felszerelés módosítása a telephelyen egy megvett fejre.
Aktorok	Takarító
Forgatókönyv	A takarító beírja az equip <plow_id> <head_id> parancsot. A gép aktív feje megváltozik, a rendszer nyugtázza.

7.3 Tesztelési terv

Teszt-eset neve	1. Pályabetöltés és teljes memóriakép ellenőrzése
Rövid leírás	Betöltünk egy előre elkészített, komplex térképet tartalmazó fájlt a load paranccsal. A fájl tartalmaz sávokat, épületeket, kereszteződéseket és járműveket is. A betöltés után a list és stat parancsokkal a teljes memóriaképet kilogoljuk a kimenetre. Az elvárt kimenetben pontosan a megadott értékeknek kell szerepelniük.
Teszt célja	A Map osztály felépítésének, a bemeneti konfigurációs parancsoknak (create, connect_fields, spawn) és az objektumok azonosító-alapú nyilvántartásának verifikálása.

Teszt-eset neve	2. Játékállapot kimentése és visszatöltése (Save/Load)
Rövid leírás	Egy módosított (már játszott) pályán kiadjuk a save parancsot, aminek eredményét egy fájlba irányítjuk. Ezt követően a programot újraindítjuk, és ezt az imént generált fájlt adjuk meg bemenetként a load parancsnak.

	A visszatöltött állapotot lekérdezve meg kell egyeznie a mentés előttivel.
Teszt célja	A kimeneti nyelv és a bemeneti nyelv kompatibilitásának, valamint a játékállapot hiánytalan szerializációjának/deszerializációjának ellenőrzése.

Teszt-eset neve	3. Busz sikeres célba érése és forduló regisztrálása
Rövid leírás	A tesztben a buszt egyenesen egy szomszédos végállomásra léptetjük a move_bus paranccsal. A kimeneten figyeljük, hogy megjelenik-e a sikeres megérkezést nyugtázó esemény. A buszvezető statisztikáját lekérdezve a teljesített fordulók számának eggyel nőnie kell.
Teszt célja	A Bus.move() metódus, a Terminal.registerArrival() hívás és a játékos (Player) pontszámítási logikájának tesztelése.

Teszt-eset neve	4. Érvénytelen lépés megkísérlése nem szomszédos mezőre
Rövid leírás	Egy járművet (pl. hókotrót) egy olyan azonosítójú sávra próbálunk léptetni, amivel nincs közvetlen összeköttetése (nincs a szomszédjai között). A rendszernek azonnal hibaüzenetet kell dobnia. A jármű lekérdezett pozíciója nem változhat meg.
Teszt célja	A gráf alapú navigáció, a Field.getNeighbors() lista ellenőrzésének és a szabálytalan mozgás kiszűrésének tesztelése.

Teszt-eset neve	5. NPC autó útvonaltervezése
Rövid leírás	Létrehozunk egy autót távol a beállított munkahelyétől. Többször kiadjuk a next_turn parancsot, szimulálva az idő múlását. Az autó pozíciójának lekérdezésekor látszódnia kell, hogy a legrövidebb úton halad a célja felé.
Teszt célja	A Car.calculatePath() algoritmus és az autonóm move() lépések helyességének vizsgálata a körökre osztott rendszerben.

Teszt-eset neve	6. NPC autó reakciója lezárt sávra
Rövid leírás	Egy haladó autó előtti sávot manuálisan lezártra (isBlocked=true) állítunk, mintha baleset történt volna. A next_turn után ellenőrizzük az autót: nem léphet rá a lezárt sávra, hanem vagy megáll, vagy új utat tervez.
Teszt célja	A járművek akadálykeresési logikájának és a Lane.isBlocked attribútum visszatartó erejének tesztelése.

Teszt-eset neve	7. Determinisztikus megcsúszás jégpáncélon

Rövid leírás	Bekapcsoljuk a determinisztikus módot (random off), majd egy járművet jéggel fedett, zúzalékmentes sávra léptetünk, ahol tartózkodik másik jármű. A járműnek garantáltan meg kell csúsznia. A sáv állapotának lekérdezésekor lezártnak kell lennie, a jármű pedig mozgásképtelenné válik.
Teszt célja	A Vehicle.slip() baleseti logika és a sávok fagyott (isFrozen) állapotának ellenőrzése véletlenszerűség nélkül.

Teszt-eset neve	8. Megcsúszás kikényszerítése és elkerülése (force_slip)
Rövid leírás	Egy síkos sávra lépés előtt kiadjuk a force_slip <id> false parancsot a járműre. A lépés után a sávnak járhatónak kell maradnia, a jármű pedig gond nélkül áthalad.
Teszt célja	A teszteléshez készített force_slip parancs működésének ellenőrzése, amivel a balesetmentes alternatív ágak is bejárhatók determinisztikus módban.

Teszt-eset neve	9. Hókotró takarítása hagyományos söprő fejjel
Rövid leírás	A hókotrót egy havas és zúzalékos sávra irányítjuk aktív söprő fejjel. A takarítás után lekérdezzük a sávot és a szomszédos sávokat. Az eredeti sávon a hónak és a zúzaléknak el kell tűnnie, a szomszédos sávokon viszont meg kell jelenniük.
Teszt célja	A SweepHead.clean() módosított logikájának tesztelése, ami immár a havat és a zúzalékot (removeGravel, addGravel) is átmozgatja.

Teszt-eset neve	10. Hókotró takarítása hóhányó fejjel
Rövid leírás	A gép hóhányó fejjel hajt rá egy vastag hóval fedett sávra. Az áthaladás után a sáv lekérdezésekor a hómennyiség lenullázódik. A szomszédos sávok paramétereit is lekérdezzük, de azok nem változhatnak meg.
Teszt célja	A ThrowerHead.clean() működésének verifikálása, amely véglegesen eltávolítja (járdára dobja) a havat a szomszédos utcák terhelése nélkül.

Teszt-eset neve	11. Hókotró takarítása jégtörő fejjel
Rövid leírás	Jégtörővel felszerelt gépet léptetünk egy fagyott (isFrozen=true) sávra. A takarítás után az isFrozen értékének false-ra kell váltania, és a jégvastagságnak el kell tűnnie.
Teszt célja	Az IcebreakerHead.clean() specifikus, fizikai jégzúzó logikájának ellenőrzése.

Teszt-eset neve	12. Sóhatás kialakítása sószóró fejjel
Rövid leírás	A hókotró aktív sószóróval végigmegy egy utcán. A sáv

	állapotlekérdezésekor látnunk kell a beállított saltEffect (sóhatás) értéket. A hókotró állapotlekérdezésében a só készletének csökkennie kell.
Teszt célja	A SaltHead.clean() üzemanyag-fogyasztásának és a sávok jégmentesítő, késleltető attribútumának (saltEffect) tesztelése.

Teszt-eset neve	13. Zúzalék szórása az úttestre
Rövid leírás	Zúzalékszóróval hajtunk egy jeges sávra. A gép zúzalékkészlete lecsökken, a sávon pedig a gravelDepth attribútum értéke nulláról nagyobbra vált. A jég továbbra is ott marad, de a sáv csúszásveszélyének inaktívvá kell válnia.
Teszt célja	Az újonnan bevezetett GravelHead.clean() logikájának és a sáv csúszásgátló funkcionalitásának ellenőrzése.

Teszt-eset neve	14. Komplex takarítás sárkány fejjel
Rövid leírás	A legerősebb, sárkány fejjel megyünk rá egy hóval és jéggel is erősen fedett sávra. Egy lépés után a sáv minden téli paraméterének (hó, jég) nullára kell esnie. A biokerozin készlet csökken.
Teszt célja	A DragonHead.clean() metódus hiánytalan olvasztó hatásának és a megfelelő üzemanyag levonásának bizonyítása.

Teszt-eset neve	15. Takarítási kísérlet kifogyott üzemanyaggal
Rövid leírás	Egy 0 készlettel rendelkező sószórót ráléptetünk egy fagyott sávra. A lépés sikeresen megtörténik (a gép halad), de a sáv paraméterei nem változnak meg, mivel nincs anyag a szóráshoz.
Teszt célja	A CleanerHead leszármazottak üzemanyag-ellenőrzésének (Empty state) és a feltételes végrehajtásnak a tesztelése.

Teszt-eset neve	16. Körök múlása és havazás szimulációja
Rövid leírás	Adott mennyiségű next_turn parancsot adunk ki, hogy triggereljük a játékba épített havazás eseményt. A sávokat lekérdezve a hóvastagság értékének globálisan, az egész térképen növekednie kell.
Teszt célja	A Map.snow() metódus időzített lefutásának és a teljes pályára kifejtett hatásának ellenőrzése.

Teszt-eset neve	17. Sóhatás idejének lejárta
Rövid leírás	Lekérdezünk egy olyan sávot, amin előzőleg sót szórtak szét (saltEffect > 0). Több next_turn parancs kiadása után az értéknek körönként csökkennie kell, majd nullára kell esnie, megszüntetve a jégvédelmet.
Teszt célja	A sávok időzített állapotfrissítésének (GameLogic.advanceTurn körléptető mechanizmus)

	tesztelése.

Teszt-eset neve	18. Felszerelés vásárlása a telephelyen
Rövid leírás	A takarító játékos egy telephelyen álló géphez megvesz egy új tisztítófejet. A tranzakció után lekérdezzük a játékos egyenlegét, aminek csökkennie kell, valamint a gép attachments listáját, amibe be kell kerülnie az új fejnek.
Teszt célja	Az IntegratedMarket.buyItem() metódus sikeres ágának és a játékos (Player) gazdasági változóinak tesztelése.

Teszt-eset neve	19. Vásárlási kísérlet fedezet nélkül
Rövid leírás	A játékos a telephelyen vásárolni próbál, de az egyenlege alacsonyabb, mint a kiválasztott termék ára. A rendszer hibaüzenetet ad, a játékos pénze nem megy mínuszba, és a terméket nem kapja meg.
Teszt célja	A vásárlás feltételeinek, a kivételes esetek lekezelésének és az integritás (nincs negatív pénz) biztosításának ellenőrzése.

Teszt-eset neve	20. Felszereléscsere megkísérlése az utcán
Rövid leírás	Az equip parancs kiadásával megpróbáljuk lecserélni a hókotró fejét, miközben az nem a HomeBase mezőn tartózkodik. A parancs hibaüzenetet eredményez, a gép aktív feje az eredeti marad.
Teszt célja	A helyszín-specifikus akciók kikényszerítésének (HomeBase.swapHead() vs sima Lane) és a biztonsági ellenőrzések tesztelése.

7.4 Tesztelést támogató segéd- és fordítóprogramok specifikálása

1. Fordító- és futtatókörnyezet (Java JDK)

- **Funkcionalitás:** Mivel a projekt Java nyelven készül, a szabványos javac (Java Compiler) parancssori eszközt használjuk a forráskód lefordítására, a tesztek futtatásához pedig a java futtatókörnyezetet (JRE).

2. Teszt-automatizáló szkript (RunAllTests.bat)

- **Funkcionalitás:** Egy saját fejlesztésű parancsfájl, amely automatizálja a teljes tesztelési folyamatot. A program:
    1. Végigiterál a tesztkönyvtárban található összes bemeneti fájlon (*_in.txt).
    2. Minden fájlra elindítja a prototípus alkalmazást, a szabványos bemenetet a fájlra irányítva.
    3. A program által generált kimenetet elmenti egy átmeneti kimeneti fájlba (*_out.txt).
    4. Meghívja az eredmény-összehasonlító programot (lásd 3. pont), és a képernyőre csak annyit ír ki tesztesetenként, hogy [PASS] (sikeres) vagy [FAIL] (hibás).

3. Eredmény-összehasonlító programok (Diff eszközök)

- **Funkcionalitás:** A generált kimenet (*_out.txt) és az előre definiált, elvárt kimenet (*_expected.txt) egybevetését végzik.

- Az operációs rendszerek beépített parancssori összehasonlítóit használjuk (Windows alatt az fc (File Compare), amelyeket a teszt-automatizáló szkript hív meg.

7.5 Napló

Kezdet	Időtartam	Résztvevők	Leírás
2026.03.19 19:00	2 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: Megállapodtunk hogy ki mit készít el, és hogy hogyan integráljuk a változtatásokat.
2026.03.20 20:00	3 óra	Kálmán	Tevékenység: változások integrálását.
2026.03.21. 18:00	3 óra	Héja	Tevékenység: Elkészítette z interfészt.
2026.03.21. 02:00	3 óra	Papp	Tevékenység: Elkészítette a bemeneti/kimeneti nyelvet.
2026.03.21. 10:00	3 óra	Grób	Tevékenység: Elkészítette a use-case-eket
2026.03.22. 14:00	3 óra	Dobronyi	Tevékenység: Elkészítette a teszt eseteket.
2026.03.22. 18:00	4 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: összeegyeztettük az egyénileg megcsinált részfeladatokat, és véglegesítettük a dokumentumot.